2024 16th International Conference on COMmunication Systems & NETworkS (COMSNETS) | 979-8-3503-8311-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/COMSNETS59351.2024.10427192

Time-Series Forecasting using Continuous Variables-Based Quantum Neural Networks

Prabhat Anand
TCS Research
Tata Consultancy Services Ltd
India
prabhat.prabhat@tcs.com

M Girish Chandra
TCS Research
Tata Consultancy Services Ltd
India
m.gchandra@tcs.com

Ankit Khandelwal
TCS Research
Tata Consultancy Services Ltd
India
khandelwal.ankit3@tcs.com

***Abstract*—Continuous Variable-based Quantum Computing (CVQC) has been developing at speed with a lot of promise in the field of quantum machine learning. It provides a direct and natural way to accommodate continuous values into the quantum computing framework. We carried out experiments on quantum simulators where we compared the results of time series forecasting on a type of continuous-variable quantum variational circuit, called CV-Quantum Neural Networks (CVQNN) for different types of time series like Energy Consumption data and stock price data. We compared their performance with a discrete variable-based variational quantum algorithm as well as with a classical Neural Network. Experiments showed that CVQNN can function just like a neural network but with a lesser number of parameters while tackling the two obstacles that are faced in qubit-based computing, which are, tackling continuous values and introducing controlled non-linearity into the circuits. We used the circuit for multi-step forecasting that performed better for a larger prediction window than one-step forecasting done iteratively on the predicted data. The resembling architecture of CVQNN with that of a neural network offers the flexibility of using a similar structure for both one-step and multi-step forecasting.**

***Index Terms*—Continuous Variable Quantum Neural Networks, Classical Optimisation, Differentiable Circuits, Time-series Forecasting, Gaussian States, Gaussian Operations.**

I. Introduction

There has been a significant shift in the way humans interact with technology due to machine learning (ML) [1]. By making a machine learn from the data, essentially tuning its parameters based on data, a number of predictions can be made or simulations can be performed or data can be generated. Recently, a new interest in the Quantum Computing (QC) community could be seen with the expectation that quantum computation can improve (construct) the current (new) ML algorithms by providing speed-ups or handling large dimensional data efficiently. These may act as sub-routines in the classical ML algorithms and greatly help them converge faster and improve their performance. And rightfully so, Quantum Machine learning (QML) based on discrete variables (DV) has made a significant leap in the last few years [2], [3]. One can see speed-ups and performance improvements in linear algebra operations from classical to quantum regime, constituting the core of machine learning tasks. Many algorithms either in full Fault-Tolerant Quantum regime [4]–[6] or Noisy Intermediate-Scale Quantum (NISQ) regime [7], [8] have shown a possible *quantum benefit*.

However, a major persisting concern with the algorithms handling classical data is to find ways to efficiently embed data in the quantum circuit and similarly retrieve classical data from the quantum circuit [9], [10]. Instead of encoding this information, which is generally continuous in nature rather than binary in qubits [11], one can encode classical information through techniques in Continuous Variable (CV) based QML. There have been efforts in CV-QML but the extent of research is not on par with DV-QML. Unlike in DVQC where a state vector would be measured with either of the discrete values while performing measurement, here, a state vector can be measured with continuous-valued spectrum [12]. Hence, one can omit a large circuitry in DVQC for indirectly obtaining continuous values either from probability distributions or expectation values. Because of the smooth accommodation of non-linearity and continuous values of data, CV-based QML promises performance enhancements over DV-based QML [13]–[16] in specific problems. Circuits performing these tasks are shallow and parameterized, hence suitable for near-term devices in which variational circuits are prominent [17].

A CV-based quantum computer can be built based on the gate model of computing with each qumode as a single system's coherent state that will depict a unit of information (Fig. 1). Certain types of efficient Gaussian and non-Gaussian gates [18] can be applied on a number of qumodes, depicting a certain kind of computation. Then, to convert the quantum information to classical, a specific set of measurements can be performed. This could be stated as our complete CV quantum computation (CVQC) paradigm. In a variational setting, the continuous values can be directly, after measurements, used in the cost function, a feature that is absent in DV-based QML techniques. Here, we tried to exploit quantum computers to show "quantum benefit" in a forecasting problem. A type of CV-based variational circuit popularly known as CV-Quantum Neural Networks (CVQNN) was used in the study. Previously, only a DV-based QNN was employed [19] and, in contrast, was trained on simulated noisy data.

To contextualize the techniques used in the paper, we have given a brief introduction to CVQNN in the next section.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:47 UTC from IEEE Xplore. Restrictions apply.

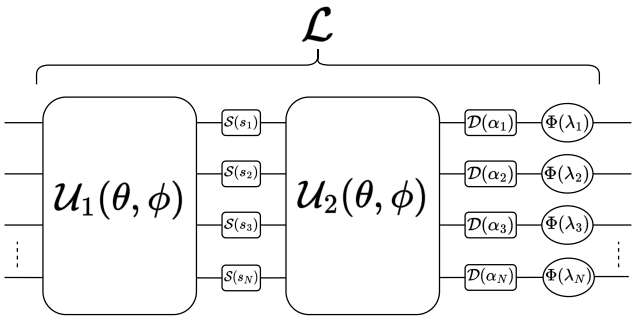


Fig. 1. A schematic representation of a single layer of a CVQNN: an interferometer comprising beam splitter gates and local rotation gates, followed by a set of squeeze gates, another interferometer, a set of displacement gates, and finally local non-Gaussian gates. The first four blocks of gates perform an affine transformation, followed by a final nonlinear transformation. Adapted from [13].

These circuits are used to solve a famous problem of time-series forecasting described in Section III. Various techniques are discussed in the Section IV. Observational results are discussed in Section V, followed by analysis and future scope in Section VI.

II. Continuous Variable Quantum Neural Networks

The architecture of CVQNN (Fig. 1) takes inspiration from classical neural networks. Neural network is one of the most successful models in classical machine learning because of their robustness as "universal function approximators" [20]. A single layer of a fully-connected NN with n neurons connected to the previous layer with m neurons represented as $\boldsymbol{x} \in \mathbb{R}^m$ can be written as

$$\mathcal{L}(\boldsymbol{x}) = \phi\big(W\boldsymbol{x} + \boldsymbol{b}\big), \qquad \mathcal{L}(\boldsymbol{x}) \in \mathbb{R}^n \tag{1}$$

where $W \in \mathbb{R}^{n\times m}$ is the weight matrix and $\boldsymbol{b} \in \mathbb{R}^n$ is the bias vector and ϕ is the non-linear activation function. Similar operations can be repeated several times to approximate as complex class of functions as required, with an arbitrary degree of accuracy [20]. This transformation can be mapped onto the CVQNN. Due to the flexibility facilitated by phase space representation [21], one can apply both affine transformation and non-linear transformations to qumodes mimicking the operations in (1). Thus, a general CVQNN is made to perform (1) by applying unitary gates from a universal gate set [12]. In particular, a layer $\mathcal{L}$ consists of the following gate sequence

$$\mathcal{L} = \boldsymbol{\Phi} \circ \mathcal{D} \circ \mathcal{U}_2 \circ \mathcal{S} \circ \mathcal{U}_1 \tag{2}$$

where:

1) $\mathcal{U}_i(\theta,\phi)$ are general N-port linear interferometers containing beam-splitters and rotations;
2) $\mathcal{D} = \bigotimes_{i=1}^{N} \hat{D}(\alpha_i)$ is a collection of displacement operators, each acting independently on each mode;
3) $\mathcal{S} = \bigotimes_{i=1}^{N} \hat{S}(s_i)$ is a collection of squeezing operators, again acting individually on each mode;
4) $\boldsymbol{\Phi} = \bigotimes_{i=1}^{N} \Phi(\lambda_i)$ is a non-Gaussian layer made up of one-mode non-Gaussian gates.

where $(\theta,\ \phi,\ s_i,\ \lambda_i, \alpha_i)$ are the real (apart from $\alpha \in \mathbb{C}$) free parameters to be optimised (Fig. 1). The biggest relief with CVQNN is that one can input the continuous-valued data by adding a layer of displacement gates [13] before the QNN layers and output can be directly obtained by measuring or evaluating expectation value. The result then would be fed to a cost function that entirely depends on the problem at hand and a classical optimizer will optimize the circuit accordingly (Fig. 2). Notice, applying the gate sequence in (2) is equivalent to performing (1), where W is applied in the singular value decomposed form ($W = U_1 S U_2$). Another contrasting feature of this architecture is that the non-linear activation function is parameterized in CVQNN, whereas it is generally a non-trainable function in classical neural networks.

III. Time Series Forecasting

In this paper, we have tried to bring out a few aspects of CV-QML and experimented with the variational algorithms [8], [13] in forecasting with real-life time series data. An important aspect of time-series analysis, that can profit from a continuous variables framework, is forecasting. It is the process of predicting future values of time-dynamical quantities based on past and present values. It has applications in Finance, Meteorology, the Energy sector, Business Operations, and Gambling to name a few [22]–[25]. There are industry-adopted classical algorithms already in practice [26].

We can define our problem more formally by considering a dynamical variable y_T whose current value can be dependent on its past values.

$$y_T = f(y_{T-1}, y_{T-2}, y_{T-3}, y_{T-4}, \cdots y_{T-n}) + \epsilon, \tag{3}$$

$$y_T = \beta_0 + \beta_1 y_{T-1} + \beta_2 y_{T-2} + \cdots + \beta_n y_{T-n} + \epsilon \tag{4}$$

where $f(.)$ is a general multi-variate function, ϵ is the arbitrary error term and β_i are trainable weights.

It is to be noted that a one-step general regression problem given in (3) is simplified to a linear regression problem in (4) as an assumption. The latter has a closed-form solution as an Ordinary Least Square (OLS) solution whose evaluation includes a matrix inversion. A recent work in variational regime [8] of DVQC presents a procedure to find this solution using an algorithm called Variational Quantum Linear Solver (VQLS).

Generalising (3) to multi-step forecasting follows directly as defining feature map F between two vectors ($\in \mathbb{R}^m, \mathbb{R}^n$) that could be written as following:

$$\begin{bmatrix} y_T \\ y_{T+1} \\ y_{T+2} \\ \cdot \\ y_{T+m} \end{bmatrix} = F\left(\begin{bmatrix} y_{T-1} \\ y_{T-2} \\ y_{T-3} \\ \cdot \\ y_{T-n} \end{bmatrix}\right) \tag{5}$$

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:47 UTC from IEEE Xplore. Restrictions apply.

The feature map in (4) and (5) can be determined by employing a CVQNN. A comparison in the performance with regards to one-step forecasting (4) can be made between CVQNN and VQLS as a benchmark process. Ultimately, we make use of CVQNN in solving multi-step forecasting, in (5), that to the best of our knowledge doesn't exist in the current literature.

IV. Methods

Since variational circuits were used whose parameters are to be learned from the data. These parameters were initialised randomly and classical optimisers were employed to optimise them. In the current scenario, these classical tasks are efficiently integrated with the quantum computing machinery on the Strawberry Fields platform (an open-source programming architecture for near-term photon-based quantum computers that can do both gradient-based and gradient-free optimisation [27]). This is particularly helpful as the optimisation process of small-scale systems is efficiently being performed on classical computing libraries based on tensor calculus. We particularly have used Strawberry Field's built-in TensorFlow backend [27] that allowed us to use Adaptive Moment Estimation optimizer (ADAM optimizer) [28] for minimising the above cost function as classical optimisation.

The model was trained on the industrial time-series data like the Energy Consumption of office space with 200 data instances [29] and average stock price data with 80 data instances of IBM (International Business Machines) [30]. We have also presented the implementation for exemplary average Yahoo stocks price data [31] converted as a time series with 100 instances.

We scaled our data before feeding them to CVQNN as we didn't want to apply large displacements to coherent states. This is due to the limitation while simulating the circuit in terms of Fock space, that one is approximating the coherent states by a finite number of Fock states by truncating the infinite series before a *cut-off* dimension. Thus, through scaling the data, the gate parameters and qumode values are prevented from taking contributions from Fock states that belong beyond the cutoff dimension.

A. Quantum One-Step Forecasting on Different Data sets

Building upon the previous work [32] in DVQC, we implemented the regression model (4) on CVQNN. Input to the circuit was a data instance encoded through displacement gates and the output data is retrieved in the form of $\langle \hat{x} \rangle$ of each qumode and represented as a vector $\hat{Y}$ where each entry of this vector is $\langle \hat{x_i} \rangle$ of ith qumode (Fig. 2), that is, $\hat{Y} = [\langle \hat{x_1} \rangle, \langle \hat{x_2} \rangle, \cdots, \langle \hat{x_m} \rangle]$. The cost function to be optimised is simply Mean Squared Error (MSE)

$$\mathcal{C} = \frac{1}{K} \sum_{j=1}^{K} \langle Y_j - \hat{Y}_j | Y_j - \hat{Y}_j \rangle, \tag{6}$$

where K is the total number of data-points. We hard-divided the data into training and testing (in a 4:1 ratio). Training the model on historical data and forecasting for future unseen values. For classical results, Numpy's [33] built-in function was employed to calculate the inverse of a matrix. For small data, it generally took milliseconds to calculate the final solution.

B. Variational Quantum Linear Solver

We compared our results from the previous section with the results obtained through VQLS for the same data set. It is also a variational method to solve a quantum system of equations. Reference [8] proposed it as a solution to HHL [4] algorithm's (introduced by Harrow, Hassidim, and Lloyd) high demand for quality computation that poses a big problem to near term devices [9].

Regression problem can be rewritten from (4) for a system in matrix form

$$X\beta = Y \tag{7}$$

where

$$X = \begin{pmatrix} 1 & x_1^1 & x_2^1 & \cdots & x_n^1 \\ 1 & x_1^1 & x_2^2 & \cdots & x_n^2 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ 1 & x_n^1 & x_2^n & \cdots & x_n^n \end{pmatrix} \tag{8}$$

$$\beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \cdots \\ \beta_n \end{pmatrix} \qquad Y = \begin{pmatrix} y_0 \\ y_1 \\ \cdots \\ y_n \end{pmatrix} \tag{9}$$

where y_i are the next value at the time step. Then, the OLS solution can be written as an equation $X^T X\beta = X^T Y$ that can be solved for β through VQLS. Essentially, a variational circuit (parametrized ansatz) is created that is optimised to give β. The optimisation of a chosen ansatz happens in a way so that $X^T X\beta'$ should be close to $X^T Y$ up to a desired accuracy, where β' is the circuit's output state vector. To find the cost function defined in [34] from the circuit's output, the Hadamard Test [8] circuit was used. The probability distribution obtained from the Hadamard Test can be used as a continuous-valued output of this discrete quantum circuit.

We used Qiskit library [35] for the implementation and used both Constrained Optimization By Linear Approximation (COBYLA) [36] (gradient-free optimisation) and Limited-memory BFGS (L-BFGS-B) [37] (gradient-based optimisation) optimiser native to the library along with state vector simulator.

C. Quantum Multi-step Forecasting

We can exploit the structure of CVQNN to get more than one continuous value for the prediction which can't be done with VQLS. Here, we will be predicting four values from the previous values.

The cost function defined here is not directly $\mathcal{C} = \langle Y - \hat{Y} | Y - \hat{Y} \rangle$ but one modified with weight matrix $\mathcal{C} = \langle Y - \hat{Y} | W | Y - \hat{Y} \rangle$ where it has a following form (for our particular choice of window)

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:47 UTC from IEEE Xplore. Restrictions apply.

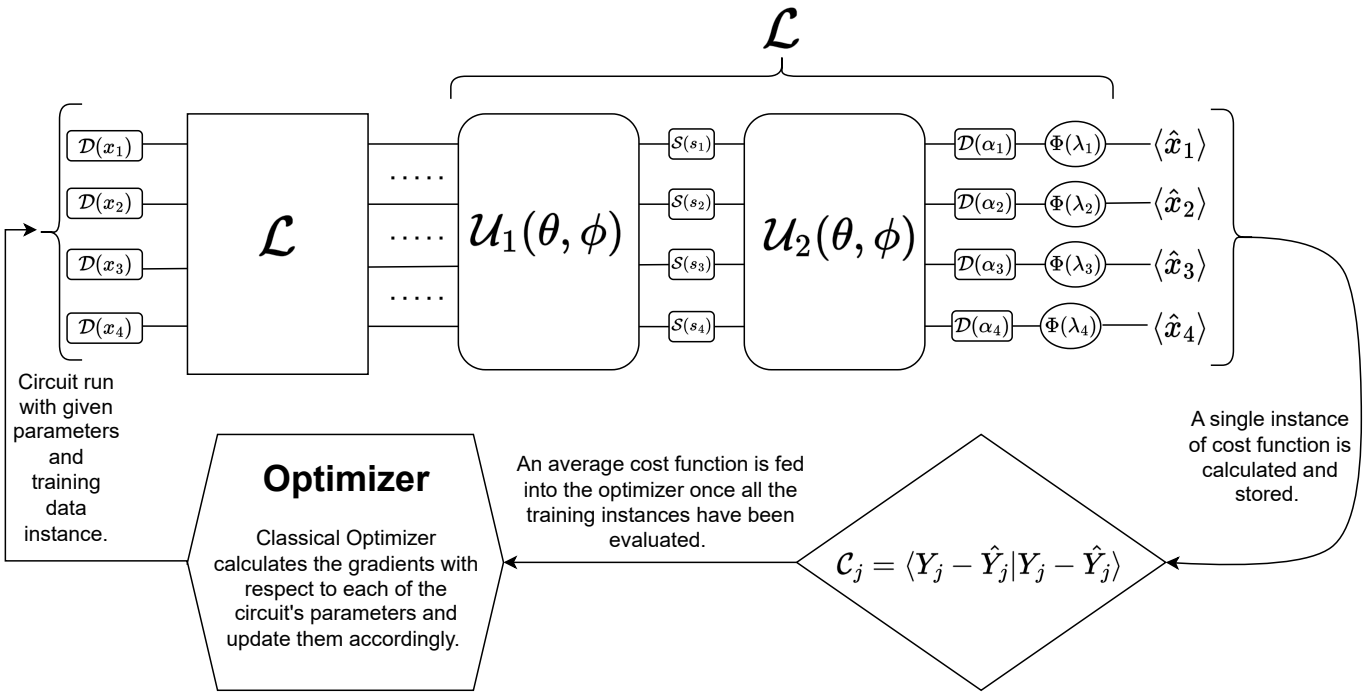


Fig. 2. Framework for CVQNN algorithm in context to multi-step forecasting: Build a model by specifying layers and initialize parameters randomly. Feed the instances of time-series data in each qumode with displacement gates after which a final state vector is obtained as an output of the layers. Evaluate cost function by directly feeding the position expectation value of each qumodes. This vectored output can be made arbitrarily close to the future time series values by defining a cost function. Optimise the circuit's parameters through a classical optimiser by minimising the cost function.

$$W = \begin{pmatrix} w_1 & 0 & 0 & 0 \\ 0 & w_2 & 0 & 0 \\ 0 & 0 & w_3 & 0 \\ 0 & 0 & 0 & w_4 \end{pmatrix} \tag{10}$$

where $\sum_i w_i = 1$ and are trainable to give the least cost for the first prediction if one wants the model to give bias to the first step in the prediction.

V. Results

Fully connected CVQNN was implemented for the task of one-step forecasting through regression. We compared the trend of overall performance with respect to an increasing number of CVQNN layers while keeping other hyperparameters like learning rate and batch size the same. The observations have been discussed in Fig. 3 and Table I. With one layer of a limited number of neurons (four in our case) a global minimum was not reached (Fig. 3), whereas with an additional layer, the same order of error was achieved that we got from classical matrix inversion.

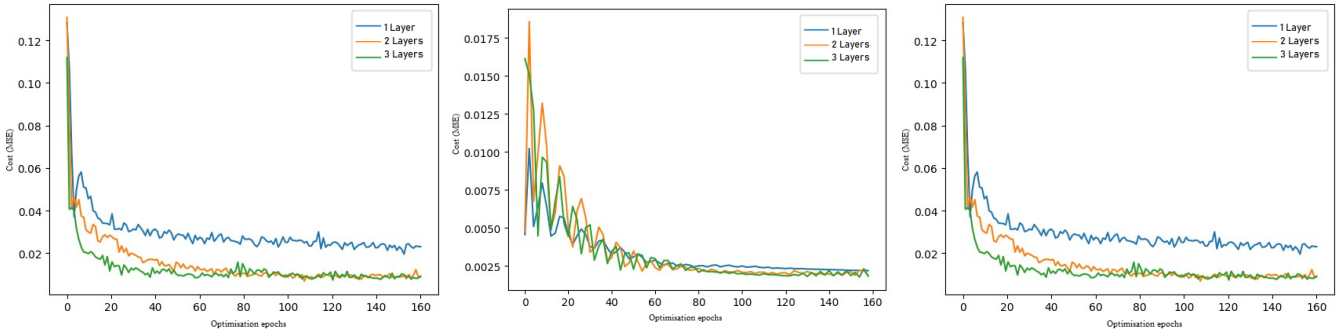
Further, we compared the accuracy of our results with DVQC based method called VQLS (Section IV-B). We found that the final loss from forecasting results of IBM stocks data was an order smaller than from the classical method. Whereas, the accuracy of forecasting results of Yahoo Stocks data and scaled Energy Consumption data were comparable to results of the classical method of direct inversion. We recorded the performance as compared to the QNN layers through the accuracy measure. We trained the CVQNNs such that the predictions made by them match the accuracy with that of VQLS (Fig. 3). The absolute error in the testing set came out to be 0.051 0.0023 0.097 for IBM stock, Yahoo stock, and Energy Consumption data set respectively. We further performed the forecasting with the same trained model iteratively. It is found to be performing poorly for predictions of later steps as the error keeps accumulating.

Thus, using multi-step forecasting has performance enhancement over iteratively using VQLS or CVQNN for one-step prediction. Also, incorporating multi-step forecasting easily into the framework of QNNs is easier in the CV model. This flexibility is provided by the architecture of CVQNN whereas extending VQLS to multi-step forecasting is not direct. Hence, we compared the forecasting results from multi-step forecasting and one-step forecasting for Yahoo data set in Fig. 4a. The error progression was averaged over the whole

TABLE I. Test Error averaged over respective data set is given for both neural networks. We note the performance of CVQNN with different layers based on testing error, training error, and convergence speed of cost function. We trained the classical neural network using the same number of trainable parameters as used in different layers of CVQNN with the same optimiser for an equal number of epochs

	CVQNN			**Classical NN**		
Layers	1	2	3	1	2	3
IBM	0.063	0.061	0.060	0.443	0.427	0.424
Energy	0.233	0.101	0.106	0.208	0.206	0.207
Yahoo	0.0022	0.0020	0.0021	0.0014	0.0013	0.0015

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:47 UTC from IEEE Xplore. Restrictions apply.



(a) IBM stocks (b) Yahoo stocks (c) Energy Consumption

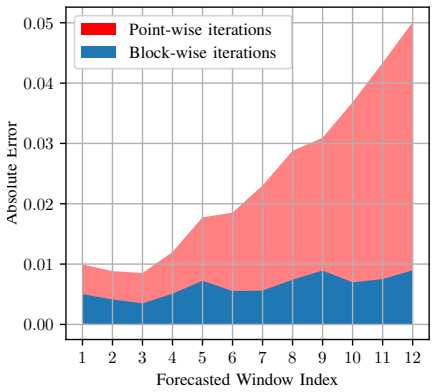
Fig. 3. Optimization results for different number of layers for three data sets. As discussed in Section IV-A, we carried out experiments to fine-tune the CVQNN architecture for specific data sets. The previous four values of time series were fed and a single value was expected as the output. Absolute error for (a) IBM stocks training data set, (b) Yahoo training data set, and (c) Energy Consumption training data is given for three different layers employed.

data set. This difference in the performance (measured through tracking absolute error) between two methods was also found in the Energy Consumption data set, though it was much less prominent.

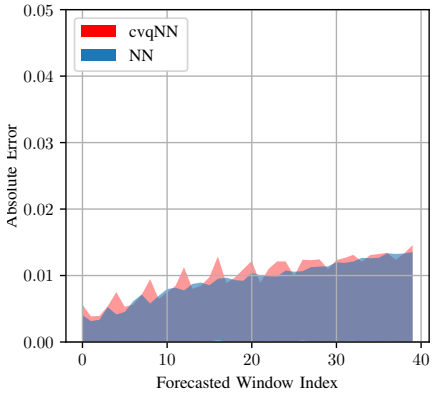
VI. Conclusion and Discussion

We verified a few of the promises made by CVQNN. They are native to continuous-valued Neural Networks and provide natural transformation states with continuous values, both linear and non-linear. From Fig. 4a, we can confirm the benefits CVQNN provides over other forecasting techniques discussed above while working on par with neural networks (Fig. 4b). Though the data set was very small in terms of the sizes of data sets that current neural networks deal with, CVQNN performed the forecasting tasks successfully on real-life data thus increasing the confidence in CVQNN. Still, due to the limited resources and exponential scaling of quantum circuits, while simulating them on classical computers, it is difficult to handle large amounts of data in the current architecture. Also, it makes it impossible to experimentally measure any speed-ups. Most notable here from Fig. 3 is the behavior of the optimization as the layer count increases which is just like a neural network. It becomes consistently easy to find a local minimum that is close to the global minimum, as the number of layers is increased. And just like the structure of a classical neural network, there is the flexibility to add any number of neurons/qumodes at any arbitrary depth of the circuit. This can be further used on current optimized circuits to cut on the cost.

Hence, one can try to encode the data or do quantum pre-processing of the data using either kernel methods or a transformation like Quantum Fourier Transform. Quantum kernel methods can be included from DVQC as they are being efficiently implemented on the state-of-the-art qubits-based machines [38], [39]. This would help in achieving a higher goal of presenting a hybrid CV-DV quantum computing machine.



(a) Comparison with point-wise iterative prediction



(b) Comparison with NN performing block-wise iterative prediction

Fig. 4. Error Progression with Yahoo Stocks data: The CVQNN model was trained on the scaled Yahoo data for four-step prediction. The optimisation used three layers of QNN instead of two and gave the final MSE to be 4.23×10^{-7}. Then we used the same model to predict the next four values from the previously predicted values and noted how the Mean Error progressed. (a) It is compared with the iterative one-step prediction using the model previously discussed. (b) Similarly, a neural network model was trained with the same number of parameters. The block-wise prediction was done iteratively and the absolute error in each prediction was recorded. A comparable performance was observed where in both cases, the error progression with block-wise prediction was minimised

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:47 UTC from IEEE Xplore. Restrictions apply.

We have kept our algorithms fully quantum without adding any classical neural network layer intentionally. Further, one can use more complex classical techniques like Long Short Term Memory (LSTM) network along with a quantum component and may achieve a performance enhancement by using these quantum circuits as subroutines [13].

Additionally, there are many classical neural networks that are based on continuous values like Continuous Boltzmann Machines and Variational Autoencoders for probability distribution generation and noise reduction. Our work points towards further exploration of these techniques, opening many paths towards a new era of computer science.

References

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015. [Online]. Available: https://doi.org/10.1038/nature14539

[2] S. Boixo *et al.*, "Characterizing quantum supremacy in near-term devices," *Nature Physics*, vol. 14, no. 6, pp. 595–600, 2018.

[3] F. Arute *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, no. 7779, pp. 505–510, 2019.

[4] A. W. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for linear systems of equations," *Physical Review Letters*, vol. 103, no. 15, oct 2009. [Online]. Available: https://doi.org/10.1103%2Fphysrevlett.103.150502

[5] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015. [Online]. Available: https://www.science.org/doi/abs/10.1126/science.aaa8415

[6] P. Rebentrost, A. Steffens, I. Marvian, and S. Lloyd, "Quantum singular-value decomposition of nonsparse low-rank matrices," *Phys. Rev. A*, vol. 97, p. 012327, Jan 2018. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.97.012327

[7] M. Benedetti, E. Lloyd, S. Sack, and M. Fiorentini, "Parameterized quantum circuits as machine learning models," *Quantum Science and Technology*, vol. 4, no. 4, p. 043001, nov 2019. [Online]. Available: https://dx.doi.org/10.1088/2058-9565/ab4eb5

[8] C. Bravo-Prieto, R. LaRose, M. Cerezo, Y. Subasi, L. Cincio, and P. J. Coles, "Variational quantum linear solver," 2020.

[9] S. Aaronson, "Read the fine print," *Nature Physics*, vol. 11, no. 4, pp. 291–293, Apr 2015. [Online]. Available: https://doi.org/10.1038/nphys3272

[10] K. Mitarai, M. Kitagawa, and K. Fujii, "Quantum analog-digital conversion," *Physical Review A*, vol. 99, no. 1, jan 2019. [Online]. Available: https://doi.org/10.1103%2Fphysreva.99.012301

[11] S. Lloyd, M. Schuld, A. Ijaz, J. Izaac, and N. Killoran, "Quantum embeddings for machine learning," 2020.

[12] S. Lloyd and S. L. Braunstein, "Quantum computation over continuous variables," *Phys. Rev. Lett.*, vol. 82, pp. 1784–1787, Feb 1999. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.82.1784

[13] N. Killoran, T. R. Bromley, J. M. Arrazola, M. Schuld, N. Quesada, and S. Lloyd, "Continuous-variable quantum neural networks," *Phys. Rev. Res.*, vol. 1, p. 033063, Oct 2019. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevResearch.1.033063

[14] M. Schuld and N. Killoran, "Quantum machine learning in feature hilbert spaces," *Phys. Rev. Lett.*, vol. 122, p. 040504, Feb 2019. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.122.040504

[15] H.-K. Lau, R. Pooser, G. Siopsis, and C. Weedbrook, "Quantum machine learning over infinite dimensions," *Phys. Rev. Lett.*, vol. 118, p. 080501, Feb 2017. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.118.080501

[16] V. Encin, "Quantum machine learning: Continuous-variable approach with application to neural networks," Oct 2020. [Online]. Available: http://webthesis.biblio.polito.it/id/eprint/15592

[17] S. Choe, "Quantum computing overview: discrete vs. continuous variable models," 2022.

[18] S. L. Braunstein and P. van Loock, "Quantum information with continuous variables," *Reviews of Modern Physics*, vol. 77, no. 2, pp. 513–577, jun 2005. [Online]. Available: https://doi.org/10.1103%2Frevmodphys.77.513

[19] D. Emmanoulopoulos and S. Dimoska, "Quantum machine learning in finance: Time series forecasting," 2022.

[20] K. Hornik, M. B. Stinchcombe, and H. L. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, pp. 359–366, 1989. [Online]. Available: https://api.semanticscholar.org/CorpusID:2757547

[21] C. Weedbrook *et al.*, "Gaussian quantum information," *Rev. Mod. Phys.*, vol. 84, pp. 621–669, May 2012. [Online]. Available: https://link.aps.org/doi/10.1103/RevModPhys.84.621

[22] C. Deb, F. Zhang, J. Yang, S. E. Lee, and K. W. Shah, "A review on time series forecasting techniques for building energy consumption," *Renewable and Sustainable Energy Reviews*, vol. 74, pp. 902–924, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1364032117303155

[23] O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu, "Financial time series forecasting with deep learning : A systematic literature review: 2005–2019," *Applied Soft Computing*, vol. 90, p. 106181, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494620301216

[24] F. Peres *et al.*, "Time series clustering of online gambling activities for addicted users' detection," *Applied Sciences*, vol. 11, no. 5, 2021. [Online]. Available: https://www.mdpi.com/2076-3417/11/5/2397

[25] Y. Ensafi, S. H. Amin, G. Zhang, and B. Shah, "Time-series forecasting of seasonal items sales using machine learning – a comparative analysis," *International Journal of Information Management Data Insights*, vol. 2, no. 1, p. 100058, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667096822000027

[26] V. I. Kontopoulou, A. D. Panagopoulos, I. Kakkos, and G. K. Matsopoulos, "A review of arima vs. machine learning approaches for time series forecasting in data driven networks," *Future Internet*, vol. 15, no. 8, 2023. [Online]. Available: https://www.mdpi.com/1999-5903/15/8/255

[27] N. Killoran, J. Izaac, N. Quesada, V. Bergholm, M. Amy, and C. Weedbrook, "Strawberry Fields: A Software Platform for Photonic Quantum Computing," *Quantum*, vol. 3, p. 129, Mar. 2019. [Online]. Available: https://doi.org/10.22331/q-2019-03-11-129

[28] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017.

[29] K. Kumar, A. Majumdar, M. Girish Chandra, and A. Anil Kumar, "Transform learning based function approximation for regression and forecasting," in *Advanced Analytics and Learning on Temporal Data*, V. Lemaire, S. Malinowski, A. Bagnall, A. Bondu, T. Guyet, and R. Tavenard, Eds. Cham: Springer International Publishing, 2020, pp. 14–25.

[30] [Online]. Available: https://www.macrotrends.net/stocks/charts/IBM/ibm/stock-price-history

[31] "Time series forecasting with yahoo stock price," Nov 2020. [Online]. Available: https://www.kaggle.com/datasets/arashnic/time-series-forecasting-with-yahoo-stock-price

[32] P. Kaushik, S. Pramanik, M. G. Chandra, and C. V. Sridhar, "One-step time series forecasting using variational quantum circuits," 2022.

[33] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: https://doi.org/10.1038/s41586-020-2649-2

[34] A. Mari, "Variational quantum linear solver," https://pennylane.ai/qml/demos/tutorial_vqls, 10 2019, date Accessed: 2023-09-08.

[35] R. Wille, R. Van Meter, and Y. Naveh, "Ibm's qiskit tool chain: Working with and developing for real quantum computers," in *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2019, pp. 1234–1240.

[36] M. J. D. Powell, "A view of algorithms for optimization without derivatives 1," 2007. [Online]. Available: https://api.semanticscholar.org/CorpusID:2570844

[37] R. Malouf, "A comparison of algorithms for maximum entropy parameter estimation," in *Proceedings of the 6th Conference on Natural Language Learning - Volume 20*, ser. COLING-02. USA: Association for Computational Linguistics, 2002, p. 1–7. [Online]. Available: https://doi.org/10.3115/1118853.1118871

[38] X. Vasques, H. Paik, and L. Cif, "Application of quantum machine learning using quantum kernel algorithms on multiclass neuron m-type classification," *Scientific Reports*, vol. 13, no. 1, p. 11541, Jul 2023.

[39] V. Havlíček *et al.*, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, Mar 2019. [Online]. Available: https://doi.org/10.1038/s41586-019-0980-2

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:47 UTC from IEEE Xplore. Restrictions apply.